

- 一、整体理解Transformer的工作流程

- 1、编码器与解码器

- 2、理解Transformer的输入与输出

- 二、Transformer的基础组件

- 1、位置编码

- 2、注意力模块

- 3、残差网络

- 4、前馈神经网络FFN和线性网络Linear

- 5、SoftMax将词表转换成概率

- 三、自注意力机制

- 1、什么是自注意力和多头注意力

- 2、深入拆解Q, K, V
 - 3、注意力的计算过程。
 - 4、Mask掩码
 - 5、多头注意力的处理
 - 6、编码器和解码器如何合作
 - 7、Transformer最后的注意点
-
- 四、Transformer与现代大模型

五、深度解析Transformer

--楼兰

这一章节我们将正式来拆开这个神秘的Transformer架构。这将是走入大模型最难的一步，但也是最坚定的一步。但在开始之前，有几件事情是你必须清楚的。

1、清楚Transformer的核心地位：了解Transformer的意义是更好的去理解现代的各种大模型，因为现代的主流大模型几乎都是由Transformer架构演进出来的。但现代的大模型又不完全等于Transformer。

2、明确学习目标：Transformer确实很重要，这并不意味着你需要详细去了解中间每一个细节的实现过程。实际上，Transformer的整个实现过程已经在Pytorch中被封装成了一个函数。手

撕Transformer在现在，纯属个人爱好。

3、找好学习方法：对Transformer的理解，要么太浅，要么太细，都不好。正确的理解要学会层层拆解！！！既不要一上来就像专业人员一样去深挖各种公式，也不要像看短视频一样，只讲功能，不讲结构。先整体，再细节，小步快跑，再回头总结。这个路线，很重要！！

如果你对手撕大模型真的感兴趣的话，可以去了解下这个项目：

<https://github.com/dabochen/spreadsheet-is-all-you-need> 用Excel手搓大模型

另外还有个GPT的可视化演示页面：<https://poloclub.github.io/transformer-explainer/>。注意，GPT就是只用了Transformer的解码器，没有用编码器。但是这个演示页面对于理解Transformer还是有帮助的。

用连连看的思路打开大语言模型的基石Transformer是什么感觉？

如果说现在有一门技术是非学不可的，那么一定是大模型了。如果在大模型中又要找出一个知识点是非学不可的，那一定是Transformer了。这东西可以说是现代整个大语言模型的基石。但这算法太过复杂，对于我们这种普通人，一点点挖掘他的算法实现，那无异于挑战攀登珠穆朗玛峰了。于是我也找了很多的科普视频来看。但是，我发现这些科普视频看完了，对transformer的问题反而更多了。要么一上来讲公式，看了等于没看。要么僻重就轻，各种酷炫的动画，莫名其妙的比喻，看着挺爽，视频一关，又不知道怎么忽视了。看了还是等于没看。

于是，这次尝试下给你分享我看Transformer的过程。用连连看的思路来拆解Transformer的每个组件。即能理解算法细节，又不丢失整体概括。虽然依然挺硬核的，但是一定能让你看完视频之后，还能想起Transformer的神奇之处。当然，如果你看完了还有问题，记得在评论区来吐槽。我会尽量为你完善其中的细节。

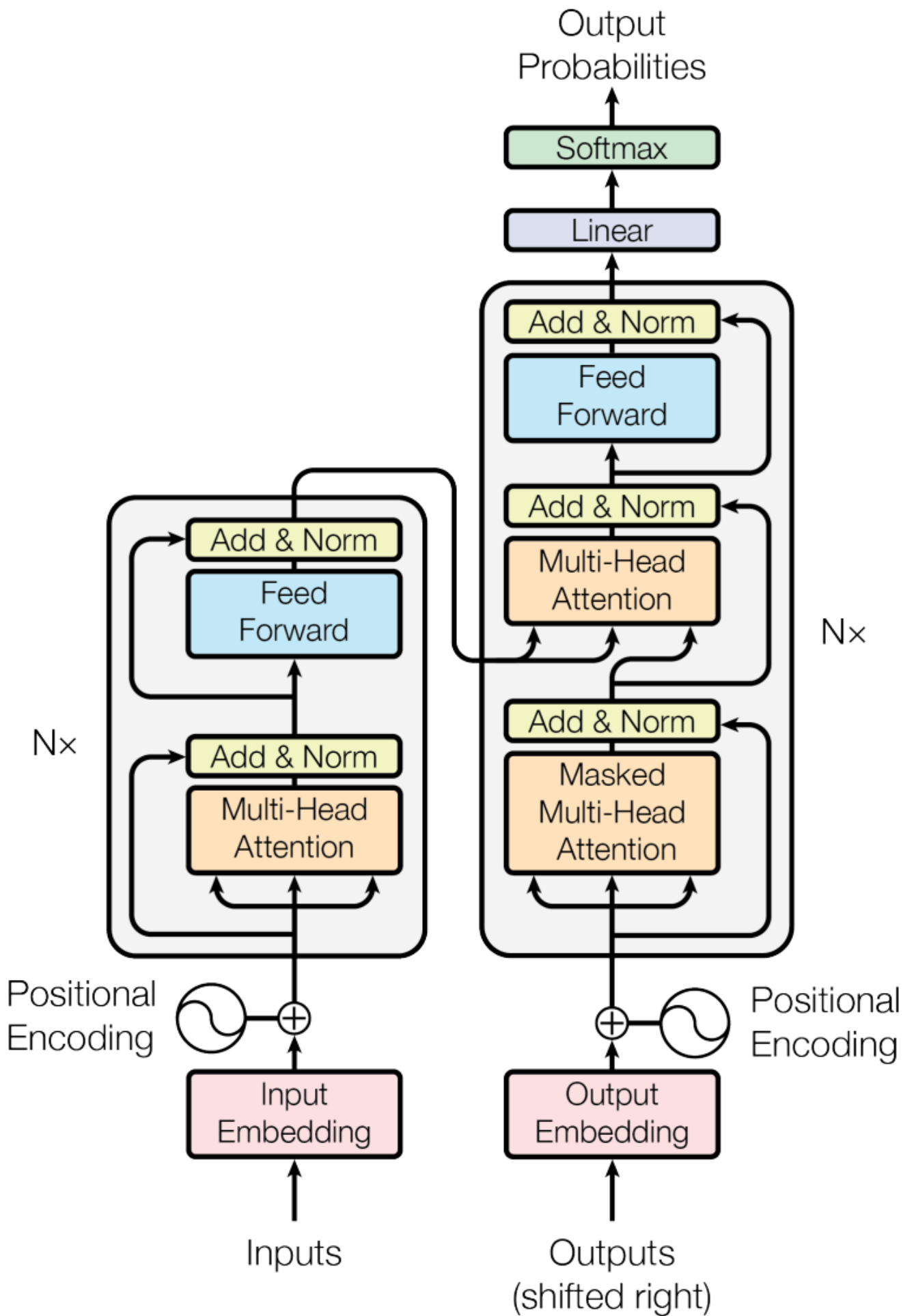
当然，记得点赞、关注、收藏。我是楼兰，IT路上一起进步。

开场补充两个重要资料，一个是Transformer的这篇经典论文《Attention is all your need》。另一个是网上大神画的这个展示图。当然，最好再结合我帮你拆解出的这些零件图。现在，暂停视频，喝口水，再深呼吸三次，Transformer走起。

一、整体理解Transformer的工作流程

1、编码器与解码器

Transformer的所有核心秘密，都在论文《Attention is all you need》中的这张图：

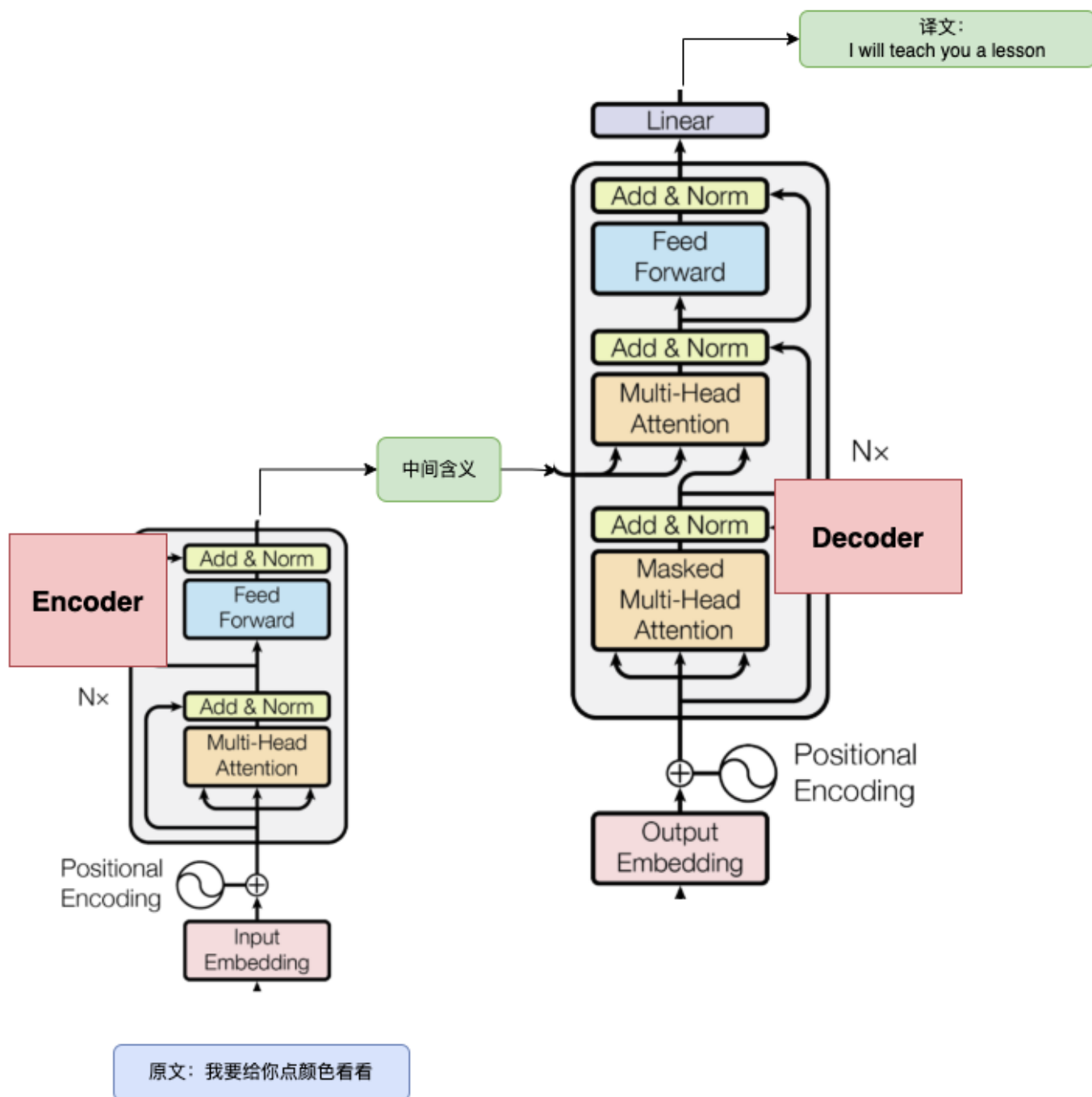


这张图怎么工作的？还得回到Transformer的最初任务来理解。

Transformer从名字就能看到，最初他是做翻译的。一个好的翻译工作应该怎么工作呢？简单的按照词去翻译或者按照语法去翻译，效果都不会太好。比如“我要给你点颜色看看”，可以翻译成“I will give you some color to see see”。每个单词都翻译到位了，语法也基本满足了，但这样明显很low。为什么？因为他并没有真正去理解语句的含义。

中文“我要给你点颜色看看”，他背后是有一些真实的语义的，表示是“我要给你点教训”。有了这个中间含义，你再去做翻译，比如翻译成“i will teach you a lesson”，这样整个翻译工作才能做到八九不离十。虽然这个中间含义并不太好用任何一种语言明确表述出来，但是他的存在就非常关键。所以一个好的翻译工作应该是这样的：原文 -(编码)->中间含义 -(解码)->译文。然后我们是要从以往的资料中学习如何进行编码和解码(这就是训练过程)，这样未来出现一个新的原文，我们就可以快速翻译成对应的译文(这就是推理过程)。

理解这个过程，那么Transformer的工作流程你也就基本上了解了。Transformer这整个架构图，左边一部分就是编码器，负责将原文(以向量表示)编码成代表中间含义的矩阵；右边一部分就是解码器，负责根据中间含义，逐步翻译成译文。把他们看成一个整体，再结合上一章节介绍的大模型的基础工作流程，整个翻译的过程就可以简化成这样：



这样一看，是不是就简单多了？

2、理解Transformer的输入与输出

以翻译“我有一只猫”这句话为例，先来理解Transformer最外层的输入和输出分别是什么。

编码器的输入与输出：

编码器的输入其实比较好理解，就是原文。不过不是原始的文本，是经过向量化后的一系列向量。原始文本经过分词后，会拆分成多个不同的文字。然后每个文字都会转化成一个固定维度的向量。这些向量叠加在一起，就成了一个矩阵。这个矩阵就是编码器的原始输入。总之，编码器的输入，就代表“我有一只猫”这个原文。总之，编码器的输入，代表用户的问题，是在用户提问的时候就已经确定下来了。

编码器的输出是什么呢？从架构图中可以看到，他的输出其实并没有一个单独的矩阵，而是会计算出几个矩阵，作为解码器的一个多头注意力模块的一部分输入。总之，编码器的输出就是代表那个不太好明确表达的中间含义。他虽然并不一定单独存在，但是会作为重要的一部分，参与后面的解码工作。

这里再做一点补充，就是关于编码器前面的分词器Tokenizer，这是一个需要在大模型训练之前提前训练的一个模型。

早期的语言模型直接用单词作为基本单位，但是这会带来一些问题：比如，语言中的单词太多了，模型词典会无限膨胀。另外，遇到一些生僻词，比如“元宇宙”这种，模型词典也无法及时收录。

于是，在每个模型前面，就会加一个Tokenizer将文本拆分成基本的语义单元，也就是Token。既能覆盖所有词汇，又能控制词典大小。

这类Tokenizer的训练是独立于大模型的前置步骤，目的不是理解语义，而是找到能高效压缩文本的子词集合。

他们只依赖文本的统计信息，和语义无关。训练好的Tokenizer和后续大模型是强绑定的。

他们拆分出来的Token，也会转成向量Embedding，作为模型的输入。但这些Embedding，并不包含语义信息。所以，是没法像RAG那样用来做语义相似度分析的。

解码器的输入与输出：

相比之下，解码器的输入就比较复杂了。要分推理和训练两个阶段来看。

首先在推理阶段，也就是要翻译新的语句。整个翻译的过程，并不是一蹴而就，而是一点点翻译出来的。解码器的输入就是从模型自己定义的特殊Token "<start of text>"开始，经过解码器一系列操作后，输出的是一个可选Token的列表。列表中会有可能的Token以及他们对应的概率。从这个可选列表中选出一个词，比如 "I"之后，就会把"<start of text>" 和 "I"，重新作为输入，再依次计算下一个可能得词。最后，直到解码器计算出另一个看不到的特殊字符 <end of text>之后，整个翻译工作才结束。并给出最终答案。

不过，在推理的过程中，解码器还会参考编码器输出的中间意思。也就是在解码器中间流程的交叉注意力环节才接入。这就像是做翻译，你眼睛盯着纸上待翻译的文本，写到一半卡住了，就抬头看一眼原文，也就是编码器的输出。找找灵感，然后再低头继续写。这两部分信息是在干活的过程中结合起来的。

然后在训练阶段，也就是解码器学习如何翻译的过程。解码器的输入是完整的译文。比如对 "我是一只猫"，他的译文是 "I have a cat"。而这个译文结果，就是解码器的输入。

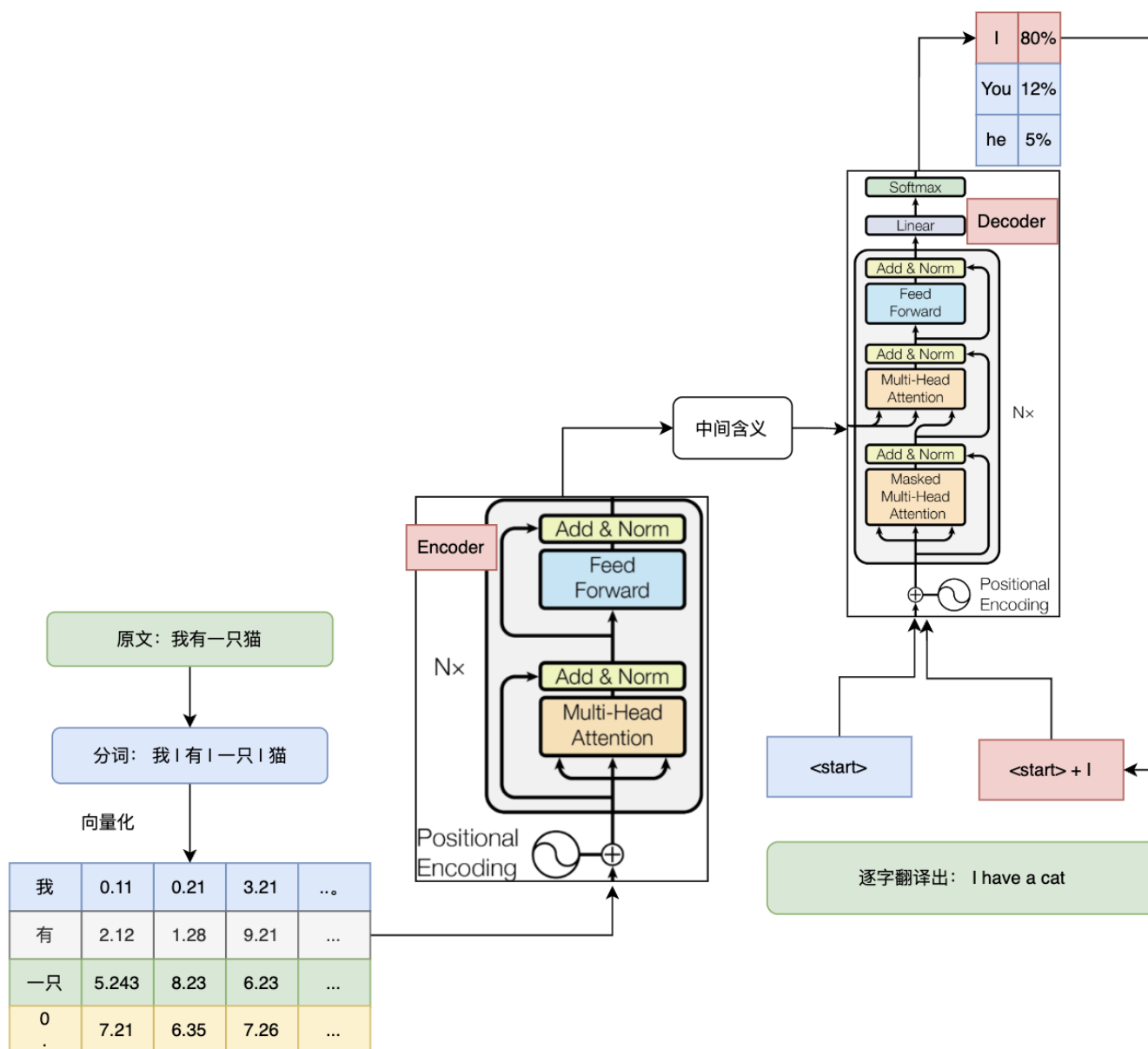
只是在学习过程中，他还是会模拟推理阶段的过程进行。只不过，对内容要做一点遮挡，也就是Mask。这里Mask分两种情况：

第一种叫Causal Mask，是防偷看的mask。这是解码器特有的。在训练时我们是知道标准答案的，但不能让模型提前偷看到后面的词，所以要把后面的词遮住，强迫他只能根据过去的词进行推测。

第二种叫Padding Mask 占位符。因为我们输入的句子有长有短，在短句的后面会补一些0进行占位，也就是padding。这些0是没有实际意义的。为了不让这些0干扰计算，我们也要用Mask把他们遮住。告诉模型：这些位置是凑数的，别理他们。

在具体实现时，就是在完整译文的向量矩阵的基础上，叠加一个新的mask遮罩矩阵，让其他部分的结果不变成没有意义的无限小。这也就是解码器中那个比编码器中多出来的Masked的意义。

把这些外围的东西加上，Transformer的工作流程就可以整理成这样：



二、Transformer的基础组件

整体理解完了，再来看里面这些细节。整个结构太复杂，那我们就一点点来插接，先去了解这里面这些大的方块是什么，然后再来看其中的连线。

1、位置编码

Positional Encoding

这个东西在解码器和编码器中都有，他的主要作用就是给输入向量增加一个位置信息。这是干嘛呢？要从形式和意义两个层面来理解。

形式上，这里所谓的位置信息，就是一个和Embedding词向量结构相同的向量，其中的数据内容主要是包含每个单词在文中的位置信息。而所谓添加的过程，就是把两个矩阵加到一起。



意义上，为什么要加位置信息呢？这跟后面的注意力机制有关。因为注意力机制是计算文本中每个词和其他词的关联性。而在这个计算过程中，对每个词的位置信息是不敏感的，也就是不区分位置。

举例来说，把“我爱你”和“你爱我”扔到注意力模块去计算，效果是差不多的。但是显然，他们的意思是完全不同的。所以就要在扔到注意力模块之前加上位置信息，把“我爱你”和“你爱我”搞点差异化出来，这样才不会混到一起。

所以很多用到注意力机制的模型，前面都会加上位置编码，这几乎是一种定式。

2、注意力模块

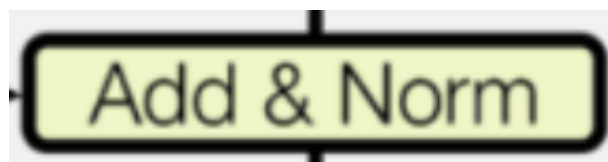


这些黄色的注意力模块是整个Transformer的核心，咱们稍后再分析细节。这里只要大概知道这些注意力模块，会把输入向量计算成一个包含了文本中每个字和其他字的关联关系的中间结果。其形式，也是一个大矩阵，只是包含的信息，会比原文本多很多。



这个模块比较复杂，也是Transformer核心中的核心。先了解整体，后面再继续深入拆解。

3、残差网络



这个东西，在每次矩阵计算后都会加上。这其实是机器学习中经常会用到的一些技巧。他的作用是防止模型训练过程中的梯度消失和梯度爆炸。

所谓梯度消失，你可以简单理解为，就是一个数字，在不断的计算过程中，越算越小，最后几乎找不到了。而他的解决办法，通常就是在每次计算后，把原来的数字再重新加上去。也就是 $y=f(x)+x$ 。这就是图中的Add残差连接。

反过来，梯度爆炸，可以简单理解为，一个数字，在不断计算过程中，越算越大，最后大到模型理解不了，甚至会导致整个模型训练失败。而他的解决办法，就是把所有数字都收缩到一定的额范文内，这就是Norm归一化。比如，讲每个数字替换成他在整个向量中占的比例，这样所有数字都变成了 $(0,1)$ 范围内的一个值。。当然，实际计算会比这个复杂很多，但整体的作用都是为了防止梯度爆炸。

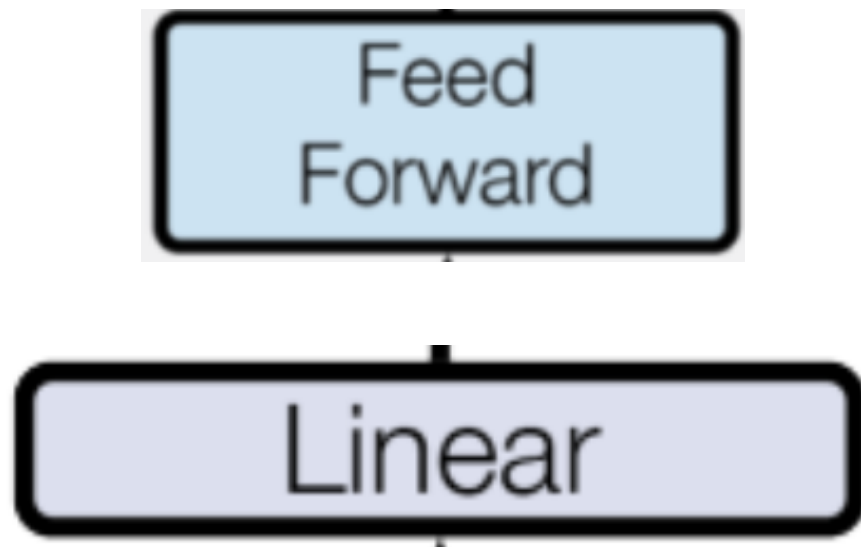
Norm层归一化的具体公式，有兴趣可以了解一下： 对于一个特征向量： $X=[x_1,x_2,...,x_d]$
(d 为特征维度)

先计算均值 $\mu = \frac{1}{d} \sum_{i=1}^d x_i$ 再计算方差 $\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$ 然后再进行归一化。 $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ 中间加入 ϵ 避免除零。

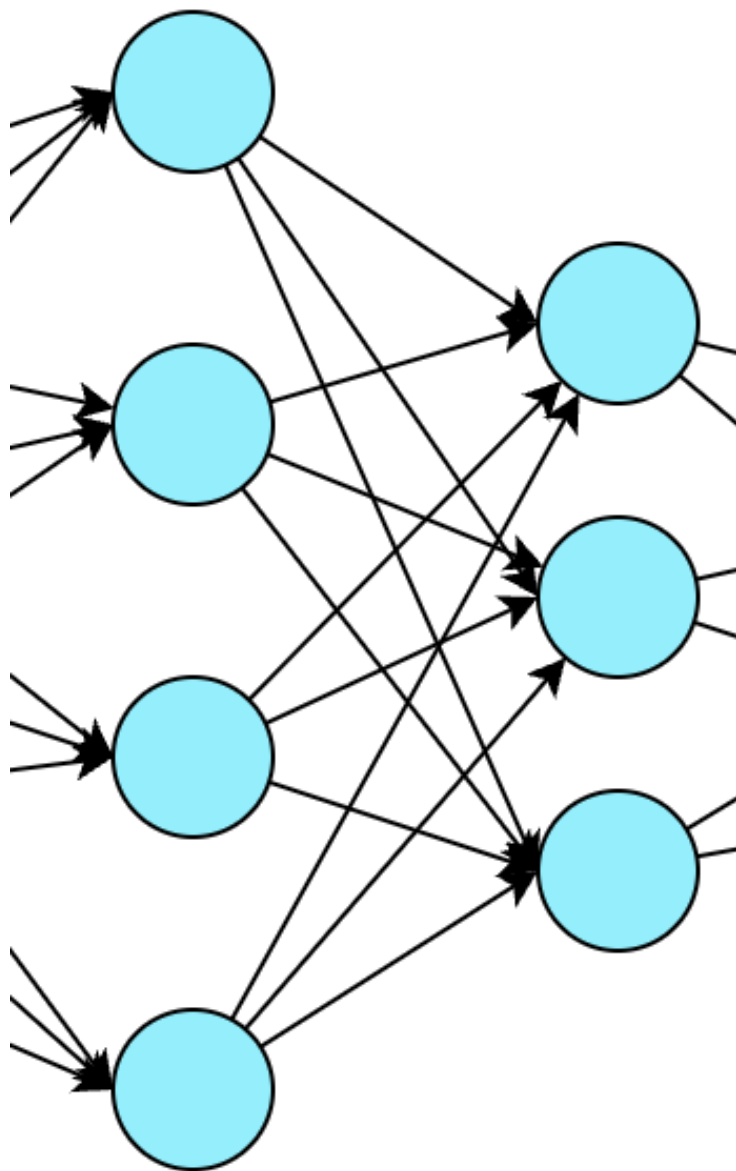
这两个东西，其实并不是Transformer独创的，在Transformer之前，人们在神经网络的研究过程中就已经形成了这样的处理方式。

对于残差网络的研究其实一直都在进行。DeepSeek最近发布了一篇文章，就设计了一种叫做 MHC流行约束超连接 的方案，其本质也就是设计一种残差网络的算法。

4、前馈神经网络FFN和线性网络Linear



这两可以一块说。他们干的事，其实都是对矩阵进行维度变化。本质上就是神经网络最典型的这种全连接层



数学公式上，就是进行一些加权求和。每个点代表一个向量，每个线代表一个权重。

先来看Linear。



在解码器的整体计算完成之后，会进行一次Linear。这个步骤主要是将解码器计算出来的隐藏语义转换为可选词表。可以简单理解为：Linear计算的输入就是解码器的语义矩阵。输出的每个节点就是一个可选Token。每个节点会计算出一个数字，这个数字就是这个Token的得分。

比如，如果输入文本是“我有一只猫”，那么Transformer要输出的结果是一个词表。里面可能有 I, You, He 等等。这个可选的词表和输入的文本个数是不一致的，所以自然就需要用Linear做升降维的操作才能匹配上。

这层Linear会对每个可选词输出一个数字Logits作为得分。但这个得分并没有太多实际意义，需要经过后续处理，才能转换成为Token的可选概率。

这个处理过程就要先经过Temperature、TopP、TopK这些参数的过滤，然后再经过SoftMax计算，最终把这些分数转换成为0-1之间的概率分布。然后就可以根据概率分布选出下一个Token了。

从这里可以看到，这个Linear权重矩阵直接决定了从语义到Token是怎么映射的，他是大模型非常核心的一种能力。所以他也是大模型训练的重点。大模型的参数有很大一部分，就集中在这个Linear层里。

然后再来看看Forward，前馈神经网络



把他和Linear放到一起，是因为他们都是基于全连接层实现的。不过他们的具体实现过程和作用是不一样的。

FFN和Linear的区别就在于，Linear只是进行简单的加权求和，FFN则要对内容做更详细的深度加工。在Transformer架构里，FFN通常会把向量先用一层Linear升维(通常是原维度的4倍)，再添加一层激活函数，引入非线性变换，这样可以处理更复杂的关系。然后再用一层Linear降维，回到原来的维度，继续进行后面计算。

所以FFN可以简单的理解为两层Linear加一层激活函数的夹心饼干。升维(Linear) -> 激活函数 -> 降维(Linear)。这个过程，输入和输出的矩阵维度是不会有变化的，所以编码器和解码器的计算过程都是可以继续叠加的。想叠加多少层就叠加多少层。

但通过FFN这个过程，就可以提炼更多语义的信息，从而让模型能够学到更复杂的语义模式。比如，有主谓关系、动宾关系等更多逻辑关系。简单来说，就是让模型理解了“这些Token组合起来表达了什么意思”。

5、SoftMax将词表转换成概率



这个SoftMax其实是机器学习中经常用到的一个通用公式。主要是对于一组输入的得分 [z1,z2,z3,z4....]转化成没给得分的概率。公式长这样：

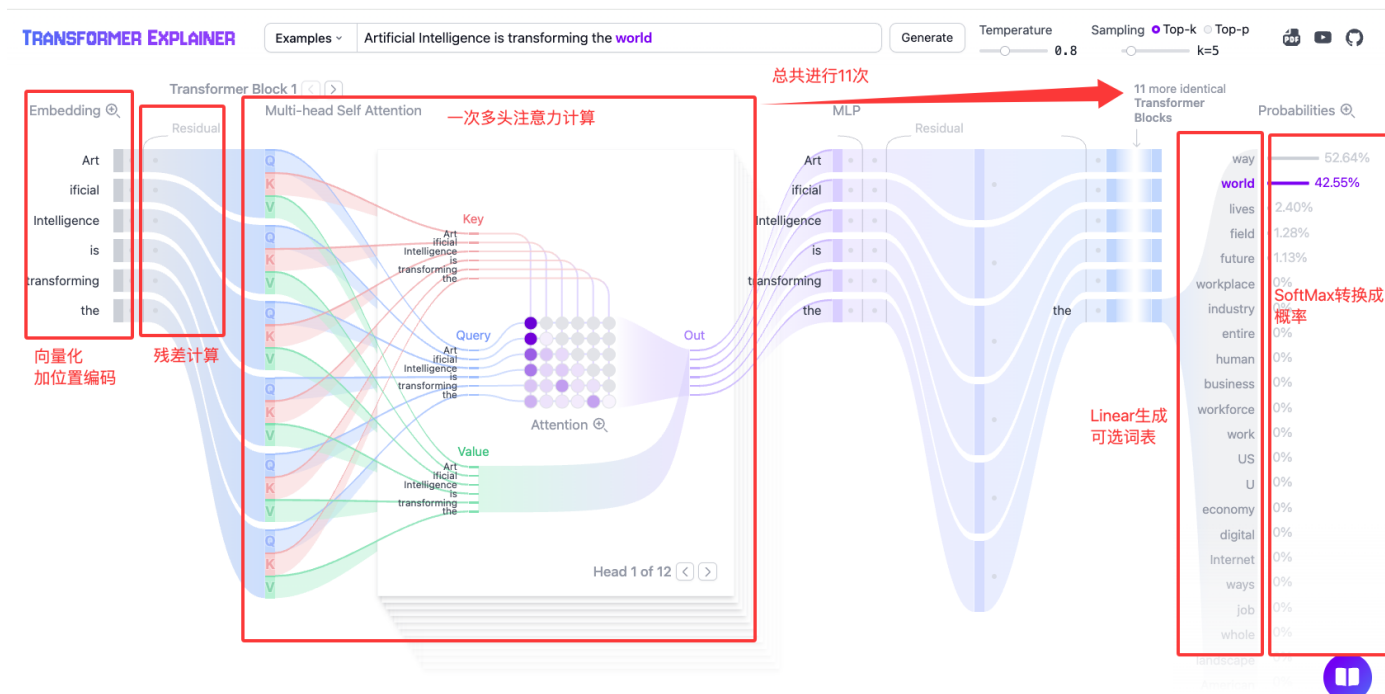
$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

到底有什么用呢？在解码器中，通过Linear线性变化，会把最终结果转化成所有候选词的得分。比如后续可能出现3万个词，那就会输出3万个数字。但是这些数字是没有意义的得分，没法直接用来选词。

通过SoftMax计算，后，就能把3万个得分转成3万个概率，这些概率加起来等于1.这样，模型就可以根据概率来选出下一个需要输出的词了。

把这些组件弄明白了，接下来整个Transformer的结构也就出来了。当然，注意结构图中有几个Nx，就是表示编码器和解码器会重复叠加好几轮。表示一次理解不够深刻，那就多理解几次。

现在，结合架构图，再来看一下之前演示的GPT的生成过程，是不是会有一点感觉？



这是GPT2的演示图，只有解码器，没有编码器。

三、自注意力机制

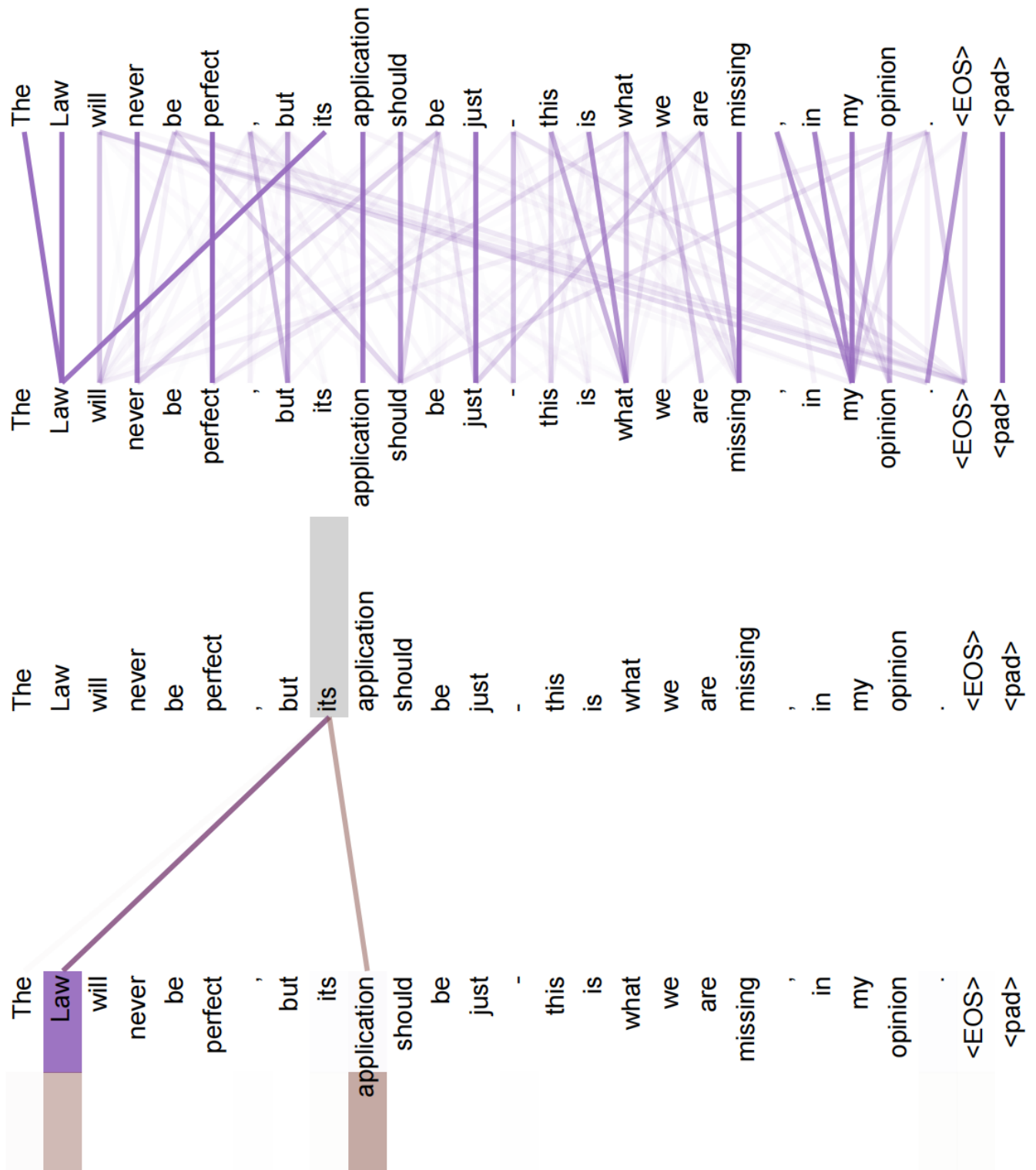
整体结构介绍完了，接下来就是硬啃这个自注意力机制了。

1、什么是自注意力和多头注意力

这个很简单，但也很关键。自注意力机制主要是用来理解文本中每个词和其他词的相关性。以翻译任务为例，找到每个词关联性最大的词，就可以更好的理解这个词要翻译成什么。

比如之前举过的例子，“我叫苹果，我喜欢吃苹果，我用苹果手机”。对于苹果这个词，在不同的位置，其他的词对他的影响力就不一样。第一个“苹果”前有个“叫”字，这个“叫”对“苹果”的影响就最大，这样就可以知道第一个苹果是名字。

就像论文中提到的这样：

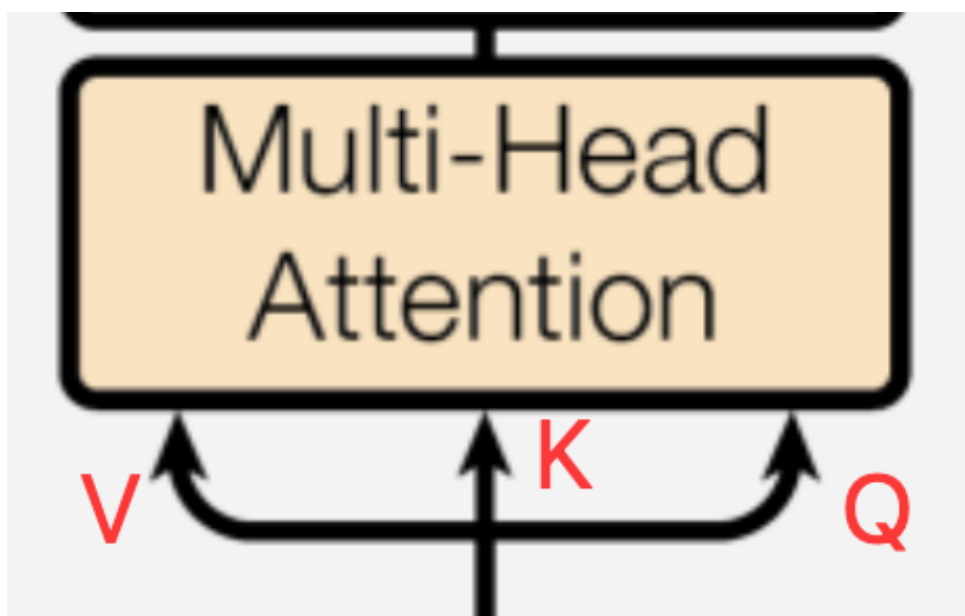


这个里面，以its这个词为例，就会分别计算和这个词关联最大的词有哪些，这样就能正确的理解its在文本中到底是什么意思。需要注意的是，

- Transformer架构中自注意力机制的计算过程是并行计算的，也就是一上来，就用分别用每个词框框一通算。这也是Transformer相比于前辈RNN很大的改进。通过并行计算，可以更好的发挥计算机的并行计算能力。
- 对一个词的理解，可能需要多次。就跟我们读课文一样的，一次没读清楚，就多读几次。每读一次，对于词的注意力也会不同。在Transformer中，每读一次就是一个头。因此就有多头注意力。

2、深入拆解Q, K, V

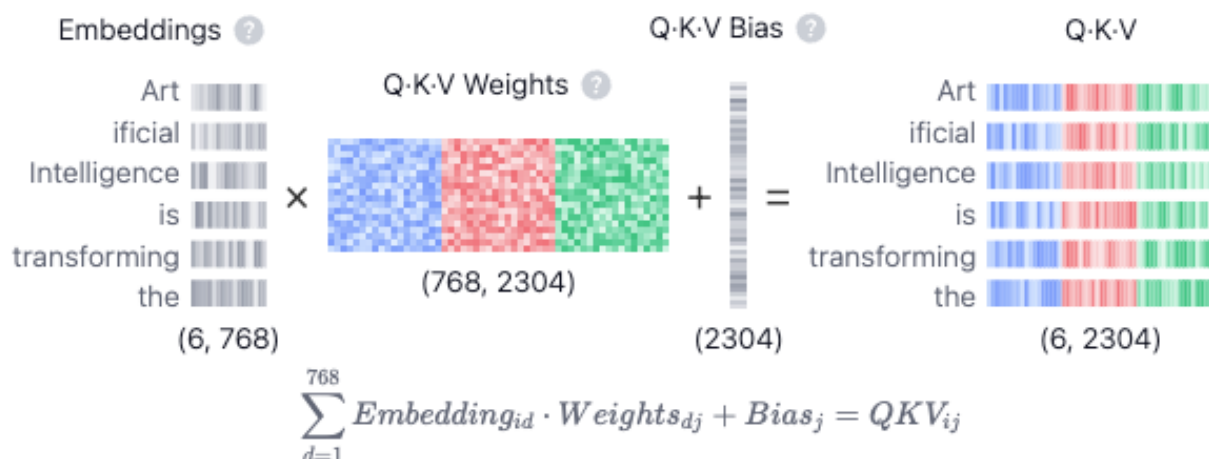
可以看到，在Transformer中，每次注意力计算都是要传入三个矩阵，分别称为Q, K, V。区别只是在不同的阶段，Q, K, V的来源不同。



这里的Q, K, V形式上，都是从原文本计算得来的三个不同的矩阵。注意，不是把原文本扔进去算，而是把原文本转化成三个不同的矩阵。

从意义上，可以有各种解读。比如把Q理解为查询，Key表示对应参考资料的目录，V表示具体的参考资料的内容。但我觉得其实没必要理解得这么透彻。因为这些矩阵背后的具体意义，反正是人类看不明白的就行了。

这里需要注意的是Q, K, V他们三个矩阵的形状。回到这个图：



看到了什么？传入的是一个6*768的原始矩阵，那么最后Q，K，V矩阵的行也一定是6。至于列数，总共是2304，也就是Q，K，V三个矩阵分别都是2304/3=768。而这个列数，跟中间那个Q，K，V的weight权重矩阵有关。

这里说明下，这个案例中是以GPT这种特定模型来介绍Q，K，V的维度的。实际上，Q，K，V的参数矩阵的维度是Transformer的一个超参数，是要根据任务来指定的。后面的各种维度也是同理。

这三个Q-K-V Weight权重矩阵的形状是固定的，都是768*768的矩阵，也就是文本的Embedding向量的维度。

至于这些矩阵中具体的值是什么呢？这就是整个Transformer要训练的超参数。也就是要通过读大量的文本，学习这三个矩阵。至于学习的过程，就是我们之前提到的反向传播。

当然，不同模型的训练过程其实是稍有不同的。比如BERT，就是按完形填空和上下文关联的方式来学习。

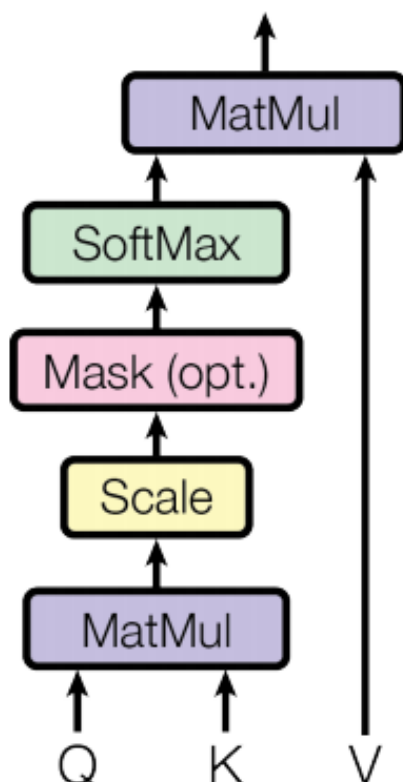
完形填空就是把原文本中某些词遮住，让模型预测这些位置的词。预测得准不准就是损失，用这个损失进行反向传播调整参数。

上下文关联就是随便拿两句话，让模型判断两句话是不是相关联的上下文。同样预测得准不准就是损失。

另外，关于三个Q-K-V Weight权重矩阵，还有一点需要补充。就是在多头注意力机制的每个头中，都有一套独立的权重参数，每组权重参数负责捕捉一种特定类型的语义关联。比如语法依赖、实体关系、逻辑连接的。至于多头怎么处理，后面马上就来。

3、注意力的计算过程。

有了Q, K, V, 那注意力机制是怎么计算的呢？



但其实这个过程，整体上就是一个不讲道理的公式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

不要问这个公式怎么来的。但就是验证下来有效果。MatMul就是在做矩阵的乘法。除以 根号 d_k ，就是在缩放Scale。softmax就是在转换概率。

稍微有点不同的就是这个Mask

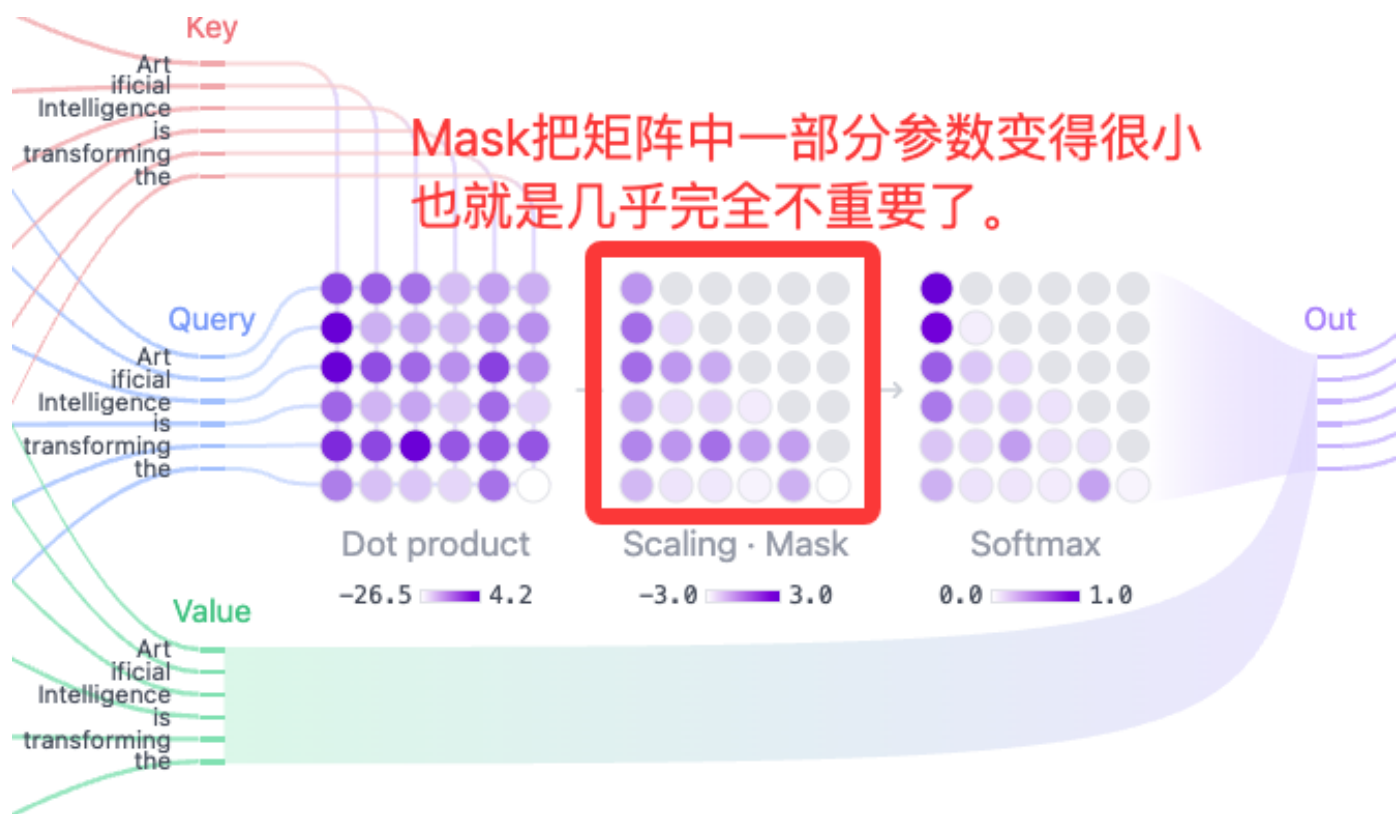
4、Mask掩码

Mask的作用之前介绍过，是在做训练时，给解码器做遮罩用的。其目的有两个，

一个是在训练阶段，遮住后面的内容，让大模型不能偷看答案，只能根据已有的内容进行预测。

另一个是进行Padding遮罩。训练时输入的句子会有长有短，这时对那些短的句子，后面会加0，进行对齐，Padding。这些0是没有实际意义的，也要进行遮罩，告诉大模型，他们没用，别理他们。

他的实现形式其实也非常简单，就是在原始矩阵的基础上，叠加一个新的矩阵，把矩阵的某些位置数字变得非常小。这样就相当于是把文本的后面一部分文字给遮起来了。



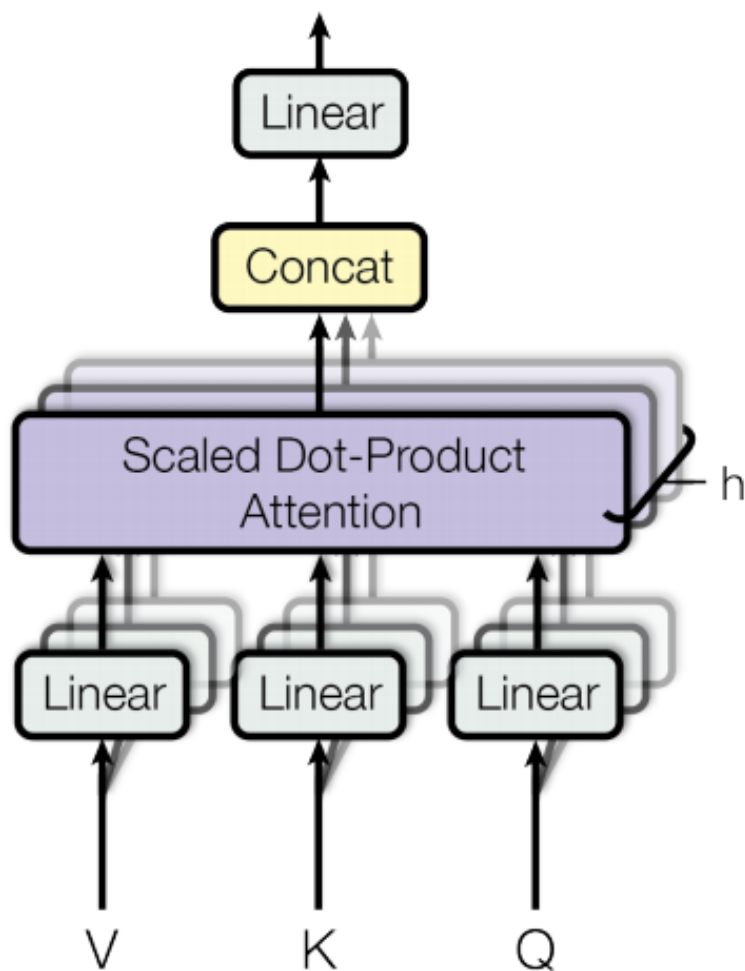
5、多头注意力的处理

单一的注意力计算可能只能关注到一种类型的相关性。而多头注意力机制就允许模型同时从多个不同的角度去理解上下文。这些不同的角度就成为“头”。

比如，一个注意力头可能专门负责“指代关系”，比如“它”指的是“猫”。另一个头可能负责“动宾关系”，比如“吃苹果”。Transformer就是通过多头计算来深度挖掘语句更深层次的关系。

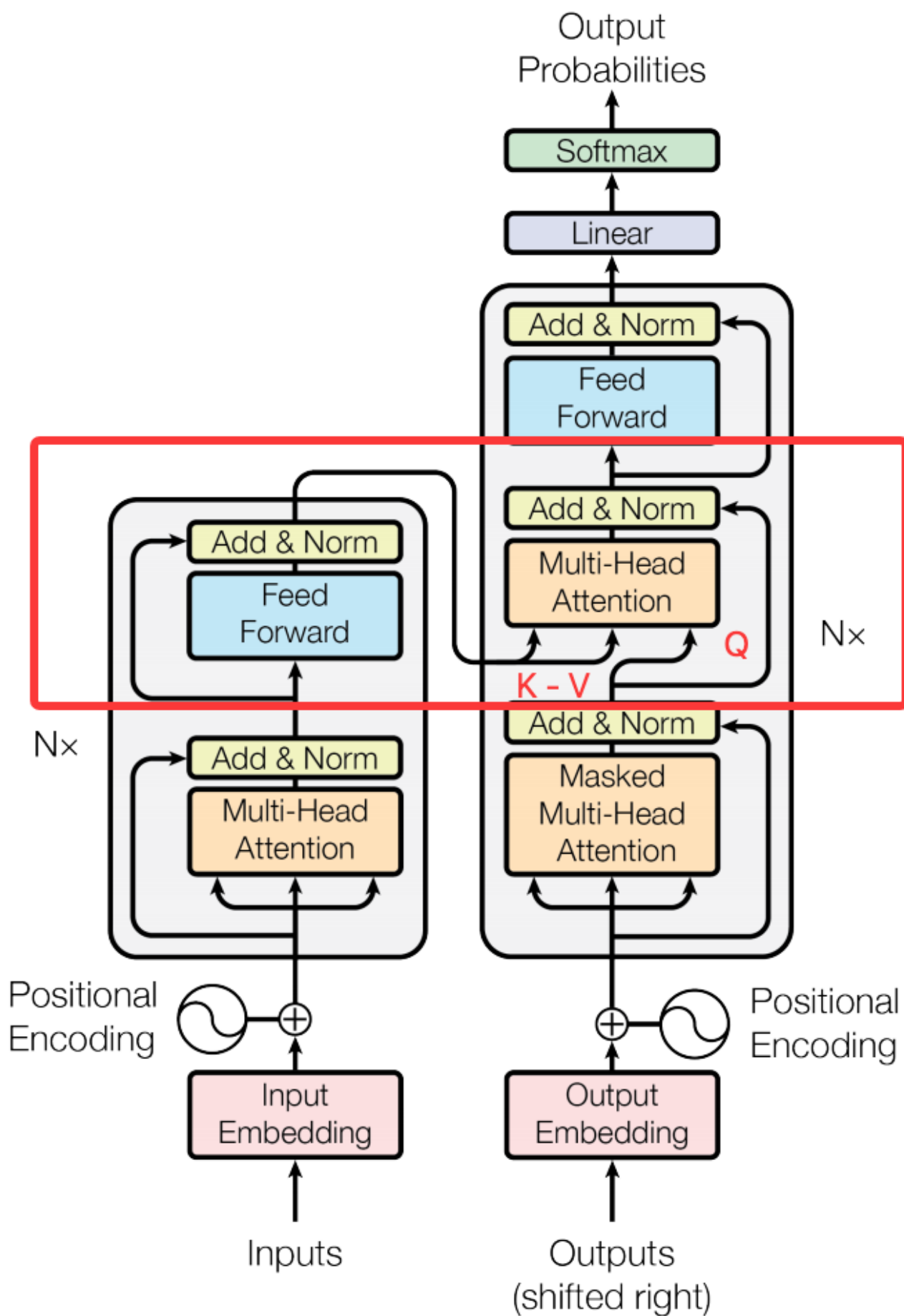
这在计算过程中是怎么处理的呢？这就需要注意注意力的输出结果。

如果是单头注意力，Q-K-V 都是768*768的矩阵，那么输出的结果就是 [6 X 768]，也就是对原文本中的每个词，都计算了他所有的注意力。但我们说了，注意力是要分多次计算出来的，也就是要计算多头注意力。像这样：



这又是怎么计算出来的呢？其实很简单。把 Q ， K ， V 分别通过Linear进行降维，平均分成 N 份（ N 就是头的数量）， N 个头并行独立的执行完整的注意力计算，之后再将 N 个头的输出结果Concat拼接到一起，再通过一次Linear恢复到原来的维度。

这里的Linear降维，就可以简单理解为平均分成多少分。有多少个头，就分成多少分。



解码器中这个来自于编码器的多头注意力计算，他的K和V就是根据编码器的输出矩阵以及 K 和V的参数矩阵计算出来的K和V矩阵。而Q就是解码器的前一步注意力计算出来的结果矩阵。

这一步的Q, K, V矩阵的形状又回到了[6 X 748] 的形状，也就是原始矩阵的形状。于是，这些注意力计算又可以继续欢乐的堆叠下去了。

而通过这个交叉计算的过程，就可以让解码器在生成每一个新词时，都能够“回顾”和“关注”输入原文的全部内容，从而进一步确保翻译的准确性和相关性。

7、Transformer最后的注意点

在原论文中，还指出了Transformer很酷的一点，就是他的注意力是可解释的。比如有的注意力头专门负责“找指代关系”（比如“它”指的是“猫”还是“垫子”），有的专门负责“找短语关系”（比如“sits on the mat”是一个整体）。就像人分析句子时的思路，机器居然也学会了“有逻辑地关注”，而不是黑箱操作。

另外，这个过程中，注意力计算分多少个头？注意力计算迭代多少次，这些参数都是超参数。也就是在任务执行时自行指定的。至于设定多少个合适，就像机器学习中很多算法一样，没有定论，只能疯狂的测试。

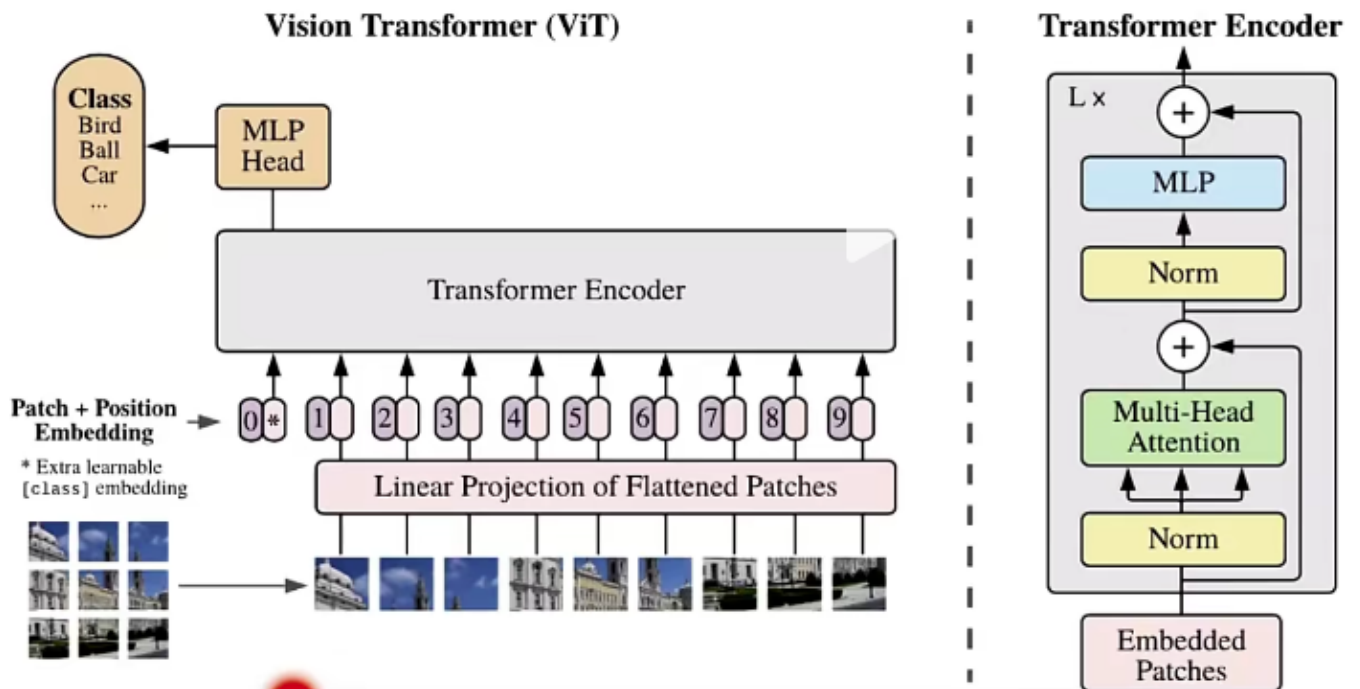
这也引出了对机器学习最底层的噩梦。一个成熟的模型，数据、算法、超参数、算力，很多因素环环相扣，甚至运气也是至关重要的一个环节。甚至在OpenAI的GPT-3模型以及ChatGPT横空出世之前，人们也想不到沿着这个道路，一直堆算力，能最终涌现出现在的大模型这么神奇的东西。所以，要想通关大模型，永远少不了一个至关重要的步骤 -- 动手实验！

四、Transformer与现代大模型

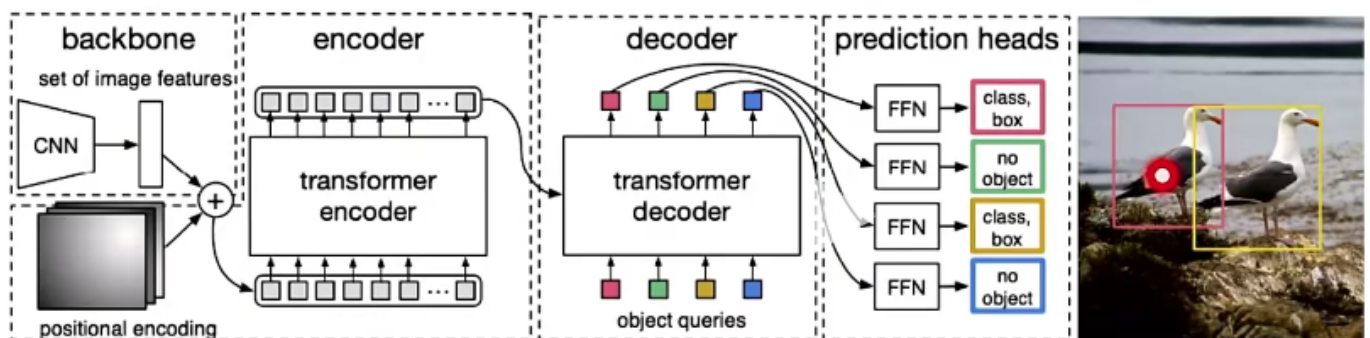
Transformer的经典之处，其实还不在模型本身，而是在于他让人们首次看到了超大规模的神经网络和超大规模并行计算的可能性，让堆算力成为一种可能。甚至Transformer的影响还不僅僅是做文字处理，在现代几乎所有主流大模型的背后，都有Transformer的影子。

- BERT：只组合的Transformer的Encoder部分，非常适合做文字理解。比如做完形填空，做阅读理解非常强。

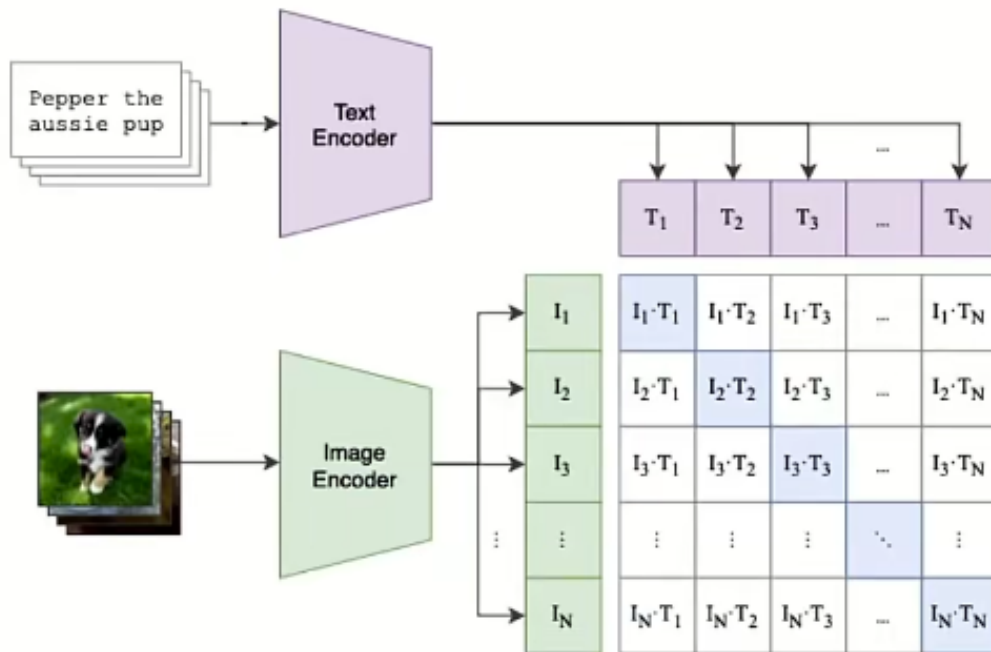
- GPT：只组合了Transformer的Decoder部分，非常适合做文字推理。由此衍生出了后续几乎所有的大语言模型。
- Vision Transformer(ViT)：做图像分类的模型。主要使用Transformer的Encoder。



- Detection Transformer(DETR)：做图形目标检测的模型



- CLIP：做多模态模型的关键模型。用于将文本和图片内容进行对齐。他对图像的处理方式就是通过ViT。



这些现代大模型，对于模型的创新，都远不止是模型层间的调整，而是从数据、训练、场景的全方面创新。